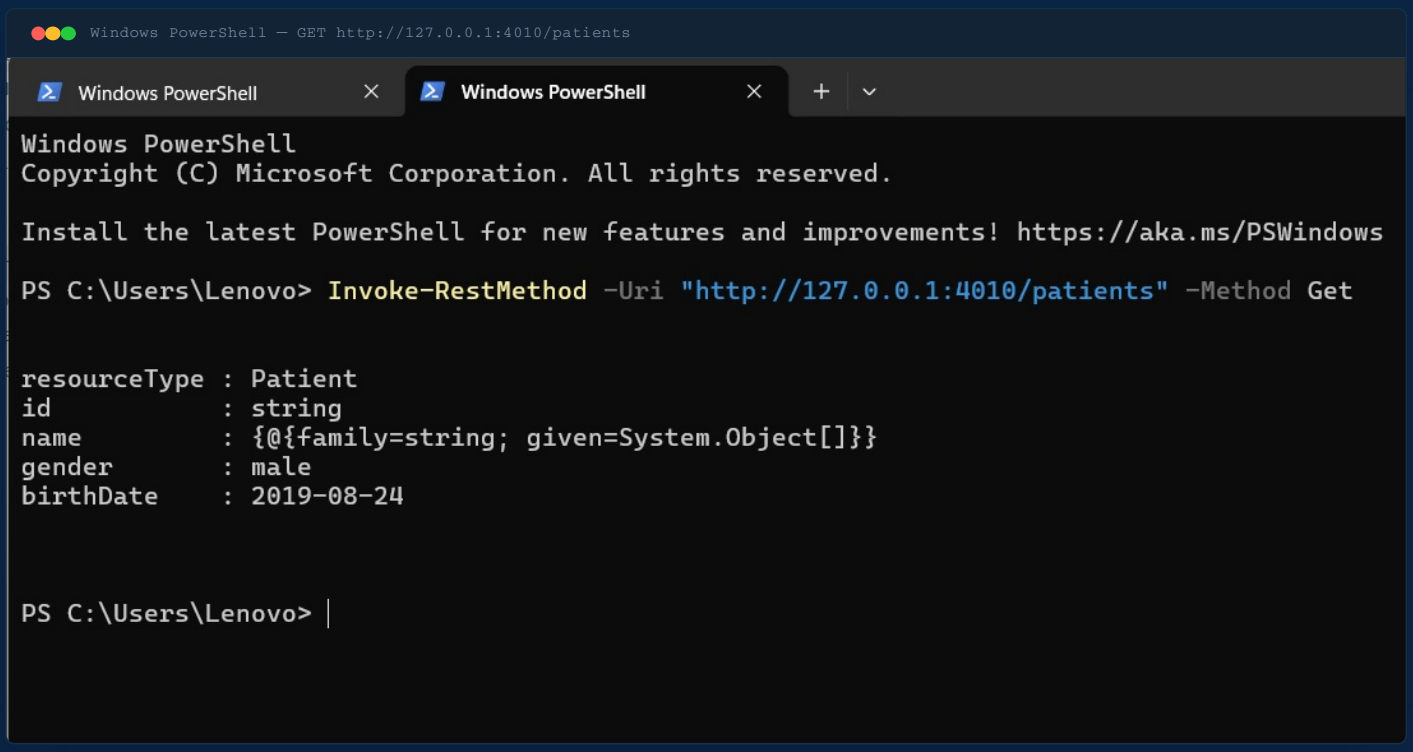
DIGITAL HEALTH · INTEROPERABILITY ENGINEERING

FHIR R4 Patient Dashboard Middleware API — Mock-First Build

A contract-first, FHIR R4-aligned REST middleware specified in OpenAPI 3.0 and validated end-to-end with a live mock server — proving the API contract before a single line of backend code is written.

OpenAPI 3.0 · FHIR R4 · Prism Mock Server · REST / JSON · LOINC · Node.js / npx · PowerShell · Invoke-RestMethod



3
FHIR R4 resources modeled end-to-end

200
OK on every live mock call

0
backend lines required to validate the contract

Kennedy Chikaodili Idoko
Digital Health & Interoperability Specialist
Berlin, Germany

GitHub github.com/kennedy216
LinkedIn linkedin.com/in/kennedy-idoko-114
Contact idokokennedy11@gmail.com

1 Project Overview

Clinical dashboards depend on a stable, well-defined data contract long before a real backend or hospital system is available to integrate against. This project demonstrates that workflow: an **OpenAPI 3.0 specification** was authored to model three core FHIR R4 resources — **Patient**, **Observation**, and **Condition** — and then brought to life as a fully working mock API using **Stoplight Prism**, so that front-end and dashboard teams can build and test against a realistic, schema-accurate contract from day one.

Problem Solved

Clinical software teams are frequently blocked waiting for a live FHIR-conformant backend. This middleware removes that bottleneck by mocking the exact response shape, status codes, and content negotiation behaviour the real API will have.

Approach

- Define resources as an OpenAPI 3.0 schema (contract-first)
- Serve the contract live via `npx @stoplight/prism-cli mock`
- Validate real HTTP calls and inspect JSON responses end-to-end

2 API Surface — Endpoints Modeled

Method	Endpoint	FHIR Resource	Status
GET	/patients	Patient — name, gender, birthDate	200 OK
GET	/observations/{patientId}	Observation — LOINC-coded vitals/results	200 OK
GET	/conditions/{patientId}	Condition — clinical & verification status	200 OK

3 OpenAPI Contract — Patient Resource

fhir-ambulatory-dashboard-api.yaml

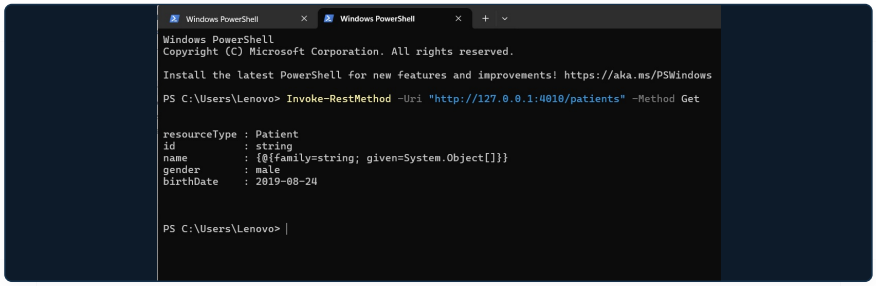
```yaml
openapi: 3.0.0
info:
  title: FHIR R4 Patient Dashboard Middleware API
  version: 1.0.0
paths:
  /patients:
    get:
      summary: Retrieve a list of FHIR Patient resources
      operationId: getPatients
      responses:
        "200":
          content:
            application/json:
              schema:
                type: array
                items: { $ref: '#/components/schemas/Patient' }
components:
  schemas:
    Patient:
      properties:
        resourceType: { type: string, example: Patient }
        name:    { type: array, items: { $ref: '#/.../Patient_name' } }
        gender:  { enum: [male, female, other, unknown] }
        birthDate: { type: string, format: date }
```

> *I successfully deployed a FHIR-compliant API middleware mock, enabling rapid prototyping for clinical dashboards. By integrating OpenAPI 3.0 specifications with Prism, I established a robust testing environment that validates clinical data structures in real time.*
>
> **— Project summary, written for LinkedIn / recruiter audience**

4 Golden Screenshots — Mock Server in Action

The contract was not just written — it was run. Below: the Prism mock server responding live to a real HTTP request, and the resulting browser-rendered JSON payloads for each of the three modeled FHIR resources.

SCREENSHOT 1 · TERMINAL

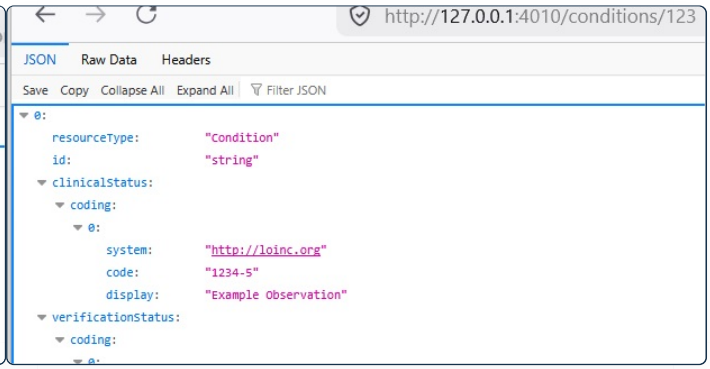


PowerShell · `Invoke-RestMethod -Uri "http://127.0.0.1:4010/patients" -Method Get` — Prism is listening on port 4010 and returns a **200** response shaped exactly to the **Patient** schema defined in the OpenAPI contract.

SCREENSHOT 2 · BROWSER — OBSERVATION

GET /observations/{patientId} — JSON response showing a LOINC-coded **Observation** resource with `category`, `code.coding`, and `valueQuantity` populated per the FHIR R4 shape.

SCREENSHOT 2 · BROWSER — CONDITION



GET /conditions/{patientId} — JSON response showing a **Condition** resource with `clinicalStatus`, `verificationStatus` and `severity` all returned as valid coded structures.

5 Skills Demonstrated

- ✓ **Contract-first API design** — modeling clinical resources in OpenAPI 3.0 before implementation
- ✓ **FHIR R4 resource modeling** — Patient, Observation, Condition with correct nested structures
- ✓ **Mock-driven development** — standing up a working API surface with Stoplight Prism / Node.js
- ✓ **Clinical terminology integration** — LOINC-coded `category` and `code` elements
- ✓ **API validation & testing** — verifying live responses via PowerShell and browser inspection
- ✓ **Recruiter-ready documentation** — translating technical evidence into a portfolio narrative

Why This Matters for Healthcare IT Roles

German digital health employers increasingly need specialists who can sit between clinical systems, compliance requirements (gemSpec_ePA, DSGVO Art. 9), and engineering teams. This project shows the ability to translate a FHIR R4 data contract into something testable and demonstrable — the same skill required to scope interoperability work for ePA, KIM, or KIS-integration projects.